

TICKETSTORM

Sistema de Reservas de Alta Concurrency

Memoria Técnica del Proyecto

Bootcamp Arquitecturas Backend a Gran Escala con Java

Autor

Matías Martínez

SRE & Backend Engineer

NFD S.A. — Paraguay

Código Facilito

Agosto de 2026

Tabla de Contenidos

1. Resumen Ejecutivo	3
1.1 Problema de negocio	3
1.2 Solución arquitectónica	3
1.3 Métricas objetivo (SLOs)	4
2. Contexto del Proyecto	5
2.1 Perfil del autor	5
2.2 Alcance del entregable	5
2.3 Estructura de la entrega	5
3. Análisis de Requerimientos	6
3.1 Requerimientos funcionales	6
3.2 Requerimientos no funcionales	6
4. Descomposición Domain-Driven	7
4.1 Bounded Contexts identificados	7
4.2 Ubiquitous Language	7
4.3 Context Map	8
4.4 Saga coreografiado	9
5. Arquitectura General	10
6. Decisiones Arquitectónicas Críticas	12
6.1 Anti-overbooking en cuatro capas	12
6.2 Fila virtual para 5M concurrentes	13
6.3 CAP híbrido por bounded context	14
7. Tech Stack Justificado	15
8. Riesgos y Mitigaciones	17
9. Roadmap y Evolución Futura	18
10. Conclusión	19

1. Resumen Ejecutivo

TicketStorm es el diseño arquitectónico de un sistema backend para venta de tickets en eventos masivos, presentado como proyecto final del Bootcamp de Arquitecturas Backend a Gran Escala con Java de Código Facilito.

El sistema debe soportar 50 millones de usuarios diarios activos (DAU), con picos de hasta 5 millones de usuarios concurrentes durante la apertura de venta de eventos masivos — el fenómeno conocido como "Taylor Swift effect" que ha llevado a colapsar sistemas de venta de tickets de proveedores establecidos globalmente. La restricción fundamental del sistema es cero overbooking: un asiento no puede venderse dos veces bajo ninguna circunstancia.

1.1 Problema de negocio

La combinación de requisitos crea una tensión arquitectónica fundamental. Por un lado, necesitamos alta disponibilidad para mostrar el catálogo a decenas de millones de usuarios simultáneamente. Por otro, necesitamos consistencia estricta cuando esos usuarios intentan reservar el mismo asiento. Estos dos requisitos son matemáticamente opuestos según el CAP Theorem.

La solución no es elegir entre uno u otro, sino diseñar un sistema híbrido donde cada bounded context toma la decisión adecuada según su rol en el negocio.

1.2 Solución arquitectónica

Se propone una arquitectura de microservicios event-driven basada en:

- Siete bounded contexts descompuestos mediante Domain-Driven Design (DDD).
- Event Sourcing en los servicios críticos (Inventory, Reservation), donde el estado es una proyección del log inmutable de eventos.
- Saga coreografiado sobre Apache Kafka para orquestar transacciones distribuidas sin acoplamiento central.
- Fila virtual (Virtual Queue) para desacoplar el spike de tráfico del sistema crítico.
- Cuatro capas independientes de defensa anti-overbooking basadas en Redis, Redisson Redlock, PostgreSQL pessimistic locking y constraints sobre el Event Store.
- Integración de Spring AI como componente arquitectónico justificado: búsqueda semántica, chatbot RAG multi-turn y fraud detection en tiempo real.

1.3 Métricas objetivo (SLOs)

Métrica	Objetivo	Justificación
Availability mensual	99.9%	Error budget de 43.8 minutos por mes.
p99 de latencia — search	< 200ms	Browse de eventos debe sentirse instantáneo.
p99 de latencia — checkout	< 500ms	Reserva + validación bajo alta concurrencia.
p99 de latencia — queue join	< 100ms	Ingreso a fila virtual sin fricción.
Overbooking rate	0%	No-negociable por diseño.
Queue WebSocket uptime	> 99.95%	Los usuarios en fila no pueden ser expulsados.

2. Contexto del Proyecto

2.1 Perfil del autor

Matías Martínez es Site Reliability Engineer (SRE) y Backend Engineer en **NFD S.A.** (Paraguay). Trabaja diariamente con Java 21, Spring Boot, GCP, OCI, Kubernetes y el stack Grafana para observabilidad. Este proyecto integra las tecnologías del bootcamp con el stack real usado en su trabajo actual, para maximizar la aplicabilidad práctica del aprendizaje.

2.2 Alcance del entregable

Este proyecto es un ejercicio de diseño arquitectónico — no incluye una implementación funcional. Los siete entregables cubren la etapa de arquitectura de solución: análisis de requerimientos, descomposición del dominio, elección de componentes, modelado de datos, especificación de APIs, justificación de trade-offs y respuestas técnicas a un cuestionario de evaluación.

2.3 Estructura de la entrega

#	Entregable	Formato
01	Diagrama de arquitectura (System Design)	HTML interactivo
02	Descomposición DDD + Context Map	HTML interactivo
03	Data Model completo	HTML interactivo
04	API Specification (OpenAPI 3.0)	HTML interactivo
05	Tech Stack + análisis de trade-offs	HTML interactivo
06	Cuestionario de evaluación	HTML interactivo
07	Video de justificación (~18 min)	YouTube / Drive

3. Análisis de Requerimientos

3.1 Requerimientos funcionales

Los requerimientos funcionales core del sistema, según los lineamientos del proyecto, son:

1. Ver eventos — listar y filtrar eventos disponibles por venue, artista, fecha, precio y ubicación geográfica.
2. Buscar eventos — búsqueda por texto libre, con capacidad semántica opcional vía Spring AI.
3. Fila virtual — gestión de acceso controlado durante aperturas de venta masivas.
4. Reservar y comprar tickets — reserva temporal de 10 minutos seguida de procesamiento de pago.

3.2 Requerimientos no funcionales

3.2.1 Escalabilidad

- 50 millones de usuarios diarios activos (DAU).
- 5 millones de usuarios concurrentes durante picos.
- Capacidad de procesar decenas de miles de reservas por segundo durante ventas populares.

3.2.2 Disponibilidad

- 99.9% de availability mensual como SLO base.
- Tolerancia a fallo de una zona (multi-zone deployment).
- Recovery Point Objective (RPO) menor a 5 minutos ante desastre regional.

3.2.3 Consistencia

- Cero overbooking — invariante de dominio absoluto.
- Cero cobros duplicados — garantizado por idempotency keys.
- Auditoría completa vía Event Sourcing.

3.2.4 Seguridad

- Autenticación OIDC (Keycloak) con JWT RS256.
- MFA obligatorio para operaciones de pago.
- mTLS entre servicios mediante Istio Service Mesh.
- Cumplimiento PCI-DSS: el backend nunca recibe datos de tarjeta en texto plano.
- Detección de fraude en tiempo real vía modelo ML sobre Vertex AI.

3.2.5 Observabilidad

- Métricas Prometheus con SLOs definidos y burn-rate alerts.
- Distributed tracing OpenTelemetry con propagación W3C TraceContext.
- Structured logging con correlación por trace_id.

4. Descomposición Domain-Driven

La descomposición del sistema en microservicios no es arbitraria. Se aplicó Domain-Driven Design (DDD) para identificar los bounded contexts a partir del análisis del dominio de negocio.

4.1 Bounded Contexts identificados

Se identificaron siete bounded contexts, clasificados según el tipo de subdominio al que pertenecen:

Bounded Context	Tipo	Responsabilidad	CAP
Event Catalog	Supporting	Gestión de eventos, venues y artistas	AP
Virtual Queue	Core	Fila virtual determinista y control de acceso	AP
Inventory	Core	Disponibilidad de asientos y anti-overbooking	CP
Reservation	Core	Reserva temporal y orquestación de compra	CP
Payment	Supporting	Procesamiento de cobros con idempotencia	CP
Notification	Generic	Envío de emails, SMS y push notifications	AP
AI Discovery	Supporting	Búsqueda semántica, RAG y fraud detection	AP

NOTA. Los tres bounded contexts marcados como Core son los que constituyen la ventaja competitiva del sistema. No se pueden tercerizar ni comprar como productos. Los Supporting son necesarios pero no diferenciadores. Notification, como Generic, podría reemplazarse con servicios externos como SendGrid o Twilio sin afectar la identidad del negocio.

4.2 Ubiquitous Language

A continuación se definen los términos clave del dominio con su significado preciso dentro de cada bounded context. El mismo término puede tener significados distintos en contextos distintos — esta es una característica deliberada del enfoque DDD.

Término	Contexto	Definición precisa
Event	Catalog	Concierto, festival o show en un venue específico con fecha y artistas definidos.
Seat	Inventory	Unidad atómica de disponibilidad; puede ser asiento numerado o entrada general.
Reservation	Reservation	Bloqueo temporal de asientos por 10 minutos para un usuario con AccessToken válido.
AccessToken	Queue	JWT emitido cuando el usuario llega al frente de la fila; TTL de 15 minutos, no renovable.
Overbooking	Inventory	Estado prohibido: más SeatReserved activos que asientos físicos. Invariante del sistema.
SaleWindow	Catalog	Ventana temporal de disponibilidad para compra. Dispara el evento SaleWindowOpened.
FraudScore	AI Discovery	Valor entre 0 y 1 del modelo ML de fraude. Umbral de rechazo: 0.8.
IdempotencyKey	Payment	UUID único por intento de pago que garantiza exactly-once en reintentos.
EventEmbedding	AI Discovery	Vector de 768 dimensiones generado por text-embedding-004 de Vertex AI.

4.3 Context Map

Las relaciones entre bounded contexts se materializan mediante tres patrones de integración según la naturaleza del acoplamiento:

- **Kafka event-driven (asíncrono):** el patrón por defecto entre contextos. Máximo desacoplamiento. Ejemplo: SaleWindowOpened emitido por Catalog es consumido por Queue para inicializar la fila.
- **gRPC / REST síncrono:** solo cuando la respuesta es necesaria en el critical path. Ejemplo: Queue Service consulta disponibilidad al Inventory Service antes de emitir un AccessToken.
- **Anti-Corruption Layer (ACL):** presente en la integración con gateways de pago externos (Stripe, PayU). Traduce el modelo externo al modelo interno del dominio Payment.

4.4 Saga coreografiado

El flujo de compra se implementa mediante un Saga coreografiado: cada servicio reacciona a eventos publicados en Kafka, sin un orquestador central. Los pasos son:

1. Queue emite AccessGranted cuando el usuario llega al frente de la fila.
2. Reservation crea la reserva temporal y emite ReservationCreated.
3. Inventory bloquea los asientos y emite SeatLocked (tras las 4 capas de defensa).
4. Payment procesa el cobro y emite PaymentProcessed o PaymentFailed.
5. Si PaymentProcessed: Reservation emite TicketPurchased, Inventory emite SeatPurchased, Notification envía el ticket.
6. Si PaymentFailed: la Saga compensatoria emite ReservationCancelled → SeatReleased → notificación al usuario.

IMPORTANTE. El Saga coreografiado tiene menor acoplamiento que el orquestado, pero es más difícil de debuggear. La solución fue invertir en observabilidad: OpenTelemetry con W3C TraceContext permite rastrear el flujo completo de una Saga a través de todos los servicios.

5. Arquitectura General

El sistema se organiza en siete capas conceptuales. Cada capa tiene una responsabilidad clara y se comunica con las adyacentes mediante protocolos bien definidos.

5.1 Capa de Clientes

La aplicación web (React SPA con SSR para SEO), la aplicación móvil (iOS/Android nativos) y las APIs de terceros consumen el sistema vía HTTPS con TLS 1.3. Los clientes también mantienen conexiones WebSocket persistentes hacia el Virtual Queue Service.

5.2 Capa Edge

- **Cloud Armor:** WAF con reglas OWASP Top 10, DDoS Adaptive Protection L7 y rate limiting por IP.
- **Cloud CDN:** cacheo de páginas de eventos, imágenes y assets estáticos con TTL configurable.
- **Global HTTP(S) Load Balancer:** distribución de tráfico con health checks y session affinity para WebSocket.

5.3 Capa API Gateway

- **Spring Cloud Gateway:** rate limiting por userId/eventId (con backend Redis), validación de JWT contra JWKS de Keycloak, circuit breakers vía Resilience4j y traffic splitting para deployments canary (95/5).
- **Keycloak:** identity provider OIDC. JWT RS256. Refresh tokens rotativos. MFA obligatorio para el scope `payments:write`.
- **Istio Service Mesh:** mTLS automático inter-servicios (zero-trust), traffic management vía VirtualService y DestinationRule, distributed tracing con inyección automática de headers W3C TraceContext.
- **OPA (Open Policy Agent):** authorization declarativa mediante políticas Rego versionadas en Git, ejecutada como sidecar via Envoy `ext_authz` filter.

5.4 Capa de Microservicios

Los siete microservicios están contruidos con Java 21 + Spring Boot 3 y se despliegan como contenedores en GKE Autopilot. Cada uno posee su propia base de datos (schema-per-service) y publica sus domain events al event backbone Kafka.

5.5 Event Backbone

Apache Kafka gestiona toda la comunicación asíncrona inter-servicios. Los topics están **particionados por eventId**, lo cual garantiza el ordenamiento de eventos dentro de un mismo evento de negocio. La configuración usa exactly-once semantics con transactional producers en Payment y Reservation,

replication factor 3 (mínimo 2 ISR para writes) y retention de 7 días para topics de eventos, con compactación para topics de estado.

Kafka Streams se emplea para agregaciones en tiempo real: tickets vendidos por segundo, revenue dashboard, alertas operacionales de negocio.

5.6 Capa de Datos

Se aplica polyglot persistence: cada bounded context elige el motor de datos más adecuado para su patrón de acceso.

Motor	Uso principal	Bounded Contexts
PostgreSQL 16	Fuente de verdad (source of truth). Event Stores. Idempotency keys.	Catalog, Inventory, Reservation, Payment
Redis Cluster 7	Cache, bitmaps, sorted sets, distributed locks, rate limiting.	Todos (uso transversal)
Elasticsearch 8	Full-text search + faceted + geo queries.	Catalog (read-side)
pgvector	Embeddings semánticos (768 dim, HNSW).	AI Discovery

5.7 Capa de Observabilidad e Infraestructura

La observabilidad se implementa con el stack estándar del equipo NFD S.A.:

- Prometheus + Grafana para métricas y dashboards.
- OpenTelemetry Java Agent para distributed tracing, exportando a Google Cloud Trace.
- Google Cloud Logging para logs estructurados JSON con export a BigQuery.

La infraestructura reside primariamente en GCP (GKE Autopilot en us-central1 multi-zona), con OCI (Oracle Cloud) como plataforma de Disaster Recovery multi-cloud aprovechando la infraestructura ya operacional en NFD S.A.

6. Decisiones Arquitectónicas Críticas

Esta sección profundiza en las tres decisiones que definen el sistema y que tuvieron el mayor peso en el análisis de trade-offs.

6.1 Anti-overbooking en cuatro capas

El overbooking en TicketStorm es **imposible por diseño, no por convención**. La estrategia es defense in depth: cuatro capas independientes, cada una capaz de prevenir overbooking por sí sola. Las capas operan en orden de latencia ascendente: rechazamos rápido en Redis antes de tocar la base de datos.

Capa 1 — Lua script atómico en Redis

La disponibilidad de asientos vive en Redis como un **bitmap por sección** (un bit por asiento: 0 = disponible, 1 = ocupado). El check-and-set se hace con un Lua script que Redis ejecuta atómicamente. Entre el GETBIT y el SETBIT no puede ejecutarse ningún otro comando. La race condition es matemáticamente imposible en esta capa.

```
local bit_value = redis.call('GETBIT', KEYS[1], ARGV[1])
if bit_value == 0 then
    redis.call('SETBIT', KEYS[1], ARGV[1], 1)
    return 1 -- éxito: asiento reservado
else
    return 0 -- falla: asiento ya ocupado, rechazar
end
```

Para un venue de 50.000 asientos, el bitmap ocupa **~6.25 KB por sección**. La latencia de la operación completa es de aproximadamente 0.5 ms.

Capa 2 — Distributed Lock (Redisson Redlock)

Antes de escribir el evento SeatReserved en PostgreSQL, se adquiere un distributed lock con Redisson implementando el algoritmo Redlock sobre tres nodos Redis independientes. El lock requiere **quórum de dos nodos**. TTL del lock: 5 segundos. Si no se puede adquirir, retornamos HTTP 503 y el cliente reintenta.

El token del lock es un UUID único por cliente, validado en el unlock vía Lua script. Esto previene el escenario en el que el cliente A libere el lock del cliente B si su operación tardó demasiado y el lock ya fue re-adquirido.

Capa 3 — Pessimistic Locking en PostgreSQL

Dentro de la transacción que persiste el evento SeatReserved, se ejecuta `SELECT FOR UPDATE` sobre la fila del asiento en la tabla de proyecciones. Esto bloquea cualquier otra transacción que intente leer-para-modificar el mismo asiento hasta que la primera committee. Esta capa cubre el caso extremadamente improbable de fallo total de Redis: el sistema sigue siendo correcto, solo con mayor latencia.

Capa 4 — UNIQUE constraint sobre el Event Store

La tabla `inventory_events` tiene una constraint `UNIQUE (aggregate_id, aggregate_version)`. Si por algún bug catastrófico dos comandos intentaran escribir la misma versión del mismo asiento, PostgreSQL rechaza el segundo con violación de constraint (código 23505). Esta es la última línea de defensa: nunca debería activarse en operación normal, pero cuando lo hace se emite una alerta crítica en Grafana para investigación de raíz.

CLAVE. El estado de cada asiento no es una columna mutable en una tabla; es la proyección del log inmutable de eventos. No existe `UPDATE seats SET status='RESERVED'` en el sistema. Esto elimina toda una clase de bugs por race conditions en updates.

6.2 Fila virtual para 5M concurrentes

El problema: cuando se abre la venta de un evento masivo, millones de usuarios golpean el sistema en segundos. Sin diseño específico, los efectos son catastróficos: thread pools saturados, DB connections agotadas, downstream services en cascada.

La solución no es "escalar más", sino **desacoplar el spike del sistema de reservaciones mediante una fila virtual**. Los usuarios no llegan directamente al checkout; llegan a una fila donde reciben un token de posición, y el sistema procesa la fila a ritmo controlado.

Fase 1 — Pre-event (T-30 min)

- HPA pre-scale del Queue Service a la capacidad esperada.
- Redis warming: bitmap de asientos pre-cargado en caché.
- CDN pre-warm de páginas de eventos en edges globales.
- Notification masiva vía GCP Pub/Sub a los usuarios interesados.

Fase 2 — Apertura (T=0)

- Cloud Load Balancer + Cloud Armor absorben el spike en el edge.
- Spring Cloud Gateway aplica rate limiting por userId.
- Queue Service ejecuta ZADD sobre Redis Sorted Set — $O(\log N)$ — aproximadamente 23 comparaciones para 5 millones de usuarios.
- Response HTTP en menos de 100 ms con posición y URL de WebSocket.

Fase 3 — Espera (T+segundos)

Y aquí está la clave: el usuario está conectado a un WebSocket que pushea su posición. **No está haciendo polling al sistema crítico**. Se soportan 5 millones de conexiones WebSocket simultáneas gracias a los Virtual Threads de Java 21 — cada conexión cuesta aproximadamente 1 KB de memoria, no megabytes como un platform thread tradicional.

Fase 4 — Turno del usuario

Cuando el worker promueve al usuario al frente de la fila, le emite un **AccessToken JWT firmado RS256 con TTL de 15 minutos**. El token es requerido en el header X-Queue-Access-Token al hacer POST /v1/reservations. Sin token válido, la request es rechazada con 401. Esto garantiza que el checkout opera siempre a ritmo controlado.

6.3 CAP híbrido por bounded context

El CAP Theorem de Eric Brewer establece que en presencia de partición de red, un sistema distribuido debe sacrificar consistencia o disponibilidad. La partición no es opcional — siempre puede ocurrir. La elección real es entre **CP** (consistente pero indisponible durante partición) o **AP** (disponible pero potencialmente inconsistente).

CLAVE. TicketStorm no es un sistema único en CAP: cada bounded context elige su posición según su criticidad de negocio. Esta es la decisión arquitectónica más importante del diseño.

Servicios CP

Consistencia es no-negociable. Preferimos rechazar requests durante particiones antes que arriesgar datos incorrectos.

- Inventory: overbooking es inaceptable. Mejor un 503 que un asiento vendido dos veces.
- Reservation: el estado de reserva debe ser consistente. Una reserva creada exitosamente debe ser visible en lecturas subsiguientes.
- Payment: exactly-once cobro es crítico. Idempotency keys garantizan consistencia incluso ante reintentos.

Servicios AP

Disponibilidad y baja latencia priorizadas. Eventual consistency es aceptable dentro de segundos.

- Event Catalog: mostrar info levemente desactualizada (5-30s lag) es aceptable.
- Virtual Queue: posición aproximada (off by 100) es aceptable.
- Notification: at-least-once delivery aceptable (mejor recibir 2 emails que ninguno).
- AI Discovery: embeddings ligeramente stale es aceptable.

La boundary CP/AP

La decisión más importante del diseño es dónde colocar la línea divisoria entre CP y AP. En TicketStorm, esa línea está **en el momento de la reserva**. Antes del checkout (browse, search, queue), el sistema es AP. Después (reserve, pay), el sistema es CP. Los usuarios pueden navegar y esperar durante una degradación de catálogo, pero no pueden iniciar transacciones inconsistentes.

7. Tech Stack Justificado

Esta sección resume la justificación de las decisiones tecnológicas más significativas. El análisis completo con trade-offs, alternativas y contras se encuentra en el entregable 05 (Tech Stack & Trade-offs).

7.1 Java 21 con Virtual Threads

Los Virtual Threads (Project Loom) son la mejora más impactante en Java de la última década. Un Virtual Thread cuesta aproximadamente 1 KB de memoria, comparado con 1 MB de un platform thread. Esto permite escribir código imperativo simple (try/catch, I/O síncrono) sin sacrificar concurrencia masiva.

Se prefiere Virtual Threads sobre WebFlux/Reactor en la mayoría de servicios (Catalog, Inventory, Reservation, Payment, Notification), donde el código de negocio es naturalmente imperativo.

Excepción: el Virtual Queue Service mantiene WebFlux porque el caso de uso (5 millones de streams WebSocket con backpressure explícito de notificaciones) sí justifica el modelo reactivo.

7.2 Kafka como event backbone

Se eligió Apache Kafka (Confluent Platform sobre GKE) por sobre GCP Pub/Sub para la comunicación inter-servicios. Kafka ofrece:

- Ordenamiento garantizado por partition key (eventId).
- Exactly-once semantics con transactional producers.
- Replay con retention configurable.
- Kafka Streams para agregaciones en tiempo real.

Sin embargo, se usa GCP Pub/Sub para el **fan-out masivo de notificaciones** (por ejemplo, envío de 5 millones de push notifications de "la venta abre en 30 minutos"), donde el auto-scaling serverless y la simplicidad operacional justifican el costo variable.

7.3 Redis Cluster para operaciones de alta velocidad

Redis 7 en Google Memorystore, con 3 masters + 3 replicas. Se utiliza para:

- Bitmaps de asientos (Inventory).
- Sorted sets de fila virtual (Queue).
- Tokens JWT de acceso (con TTL estricto).
- Distributed locks Redlock (Redisson).
- Rate limiting con sliding window.
- Cache-aside del Catalog Service.

IMPORTANTE. Redis es in-memory. Si el cluster falla, se pierde estado transitorio (fila, caché). Se mitiga con AOF persistence, replica sync automático, y el hecho de que PostgreSQL siempre es el source of truth. Redis es speed layer, no fuente de verdad.

7.4 Spring AI como abstracción para el componente de IA

La integración con inteligencia artificial se hace mediante **Spring AI 1.x**, que estandariza la interacción con LLMs, embeddings y vector stores. El proveedor concreto es **Vertex AI Gemini 1.5 Pro** (alineado con el resto de la infraestructura GCP), pero cambiar mañana a Claude o GPT-5 requeriría solo cambiar el starter en Maven — la lógica de negocio (ChatClient, advisors, VectorStore) permanece intacta.

Las tres capacidades integradas son:

1. Búsqueda semántica: query embedding + pgvector similarity search.
2. Chatbot RAG multi-turn: ChatMemory + QuestionAnswerAdvisor + Gemini Pro.
3. Fraud detection en tiempo real: modelo custom en Vertex AI Endpoint invocado desde Payment Service.

7.5 Observabilidad con stack open source

El stack de observabilidad reutiliza el que ya opera NFD S.A. en su día a día: Prometheus + Grafana + OpenTelemetry. Esto asegura continuidad operacional y aprovechamiento del expertise interno.

Los SLOs están definidos desde el día uno, y las alertas usan multi-window burn rate (recomendación de Google SRE Book): fast burn a 1 hora (alerta si consume 2% del budget), slow burn a 3 días (alerta si tendencia consume 10%). Esto detecta tanto degradaciones rápidas como lentas.

8. Riesgos y Mitigaciones

Se identificaron los siguientes riesgos técnicos y de negocio, con sus respectivas estrategias de mitigación.

Riesgo	Impacto	Mitigación
Fallo del cluster Redis durante apertura de venta	Alto — pérdida de fila virtual	AOF persistence habilitado + replica sync + fallback a PostgreSQL para inventario.
Gateway de pago externo lento o caído	Alto — bloqueo de checkout	Circuit Breaker Resilience4j + fallback a cola async + status polling.
Poison message en un topic Kafka	Medio — pausa de procesamiento	Dead Letter Topics + alertas Grafana + retry limitado con dedup.
Scalpers automatizados usando bots	Alto — mala experiencia + reputación	reCAPTCHA Enterprise + device fingerprinting + fraud AI + hard limit por usuario.
Fallo regional de GCP us-central1	Crítico — pérdida de servicio	DR multi-cloud en OCI con replica asíncrona (RPO < 5 min).
Overbooking por bug de aplicación	Crítico — pérdida de credibilidad	4 capas independientes de defensa (Lua + Redlock + FOR UPDATE + UNIQUE).
Vertex AI Gemini con latencia alta	Bajo — degrada búsqueda semántica	Cache de embeddings recientes en Redis + fallback a Elasticsearch full-text.
Explosión de costo de Kafka en pico	Medio — presión sobre unit economics	Uso híbrido Kafka + Pub/Sub para fan-out serverless.

9. Roadmap y Evolución Futura

El diseño actual cubre el alcance del bootcamp. Se identifican las siguientes extensiones naturales para evolución futura del sistema:

9.1 Fase 2 — Escalado global

- Migración de PostgreSQL a Cloud Spanner para consistencia externa y escalado horizontal automático en operaciones globales.
- Multi-región activa: réplicas activas en 3 continentes con enrutamiento por latencia.
- Migración de pgvector a Vertex AI Vector Search si el volumen de embeddings supera los 10 millones.

9.2 Fase 3 — Personalización y AI avanzado

- Sistema de recomendaciones personalizadas basado en historial de compras + embeddings de comportamiento.
- Fraud detection con modelo self-hosted (fine-tuning sobre datos propios) para reducir dependencia de Vertex AI.
- Análisis de sentimiento en reviews de eventos mediante LLM.

9.3 Fase 4 — Optimización operacional

- Migración de Notification Service a GraalVM Native Image para reducir startup y memoria.
- Adopción de Structured Concurrency estable (Java 25 LTS) en fan-out crítico.
- Automatización de capacity planning basada en histórico de ventas.

10. Conclusión

TicketStorm demuestra un diseño arquitectónico completo capaz de resolver un problema de negocio real: la venta de tickets en eventos masivos con las restricciones de escala y consistencia más agresivas de la industria.

El proyecto integra los tópicos cubiertos en el Bootcamp de Arquitecturas Backend a Gran Escala con Java — microservicios, DDD, Event Sourcing, CQRS, Saga, análisis CAP, resiliencia, observabilidad, alta concurrencia — y agrega valor con la integración arquitectónica de Spring AI como componente justificado.

Las decisiones más significativas del diseño son:

1. El anti-overbooking implementado en cuatro capas independientes de defensa, haciendo el overbooking matemáticamente imposible.
2. La fila virtual desacoplada que permite absorber 5 millones de usuarios concurrentes sin degradar el sistema crítico.
3. La decisión CAP híbrida por bounded context, con la boundary CP/AP colocada deliberadamente en el momento de la reserva.
4. El Saga coreografiado sobre Kafka, que ofrece consistencia distribuida sin acoplamiento central.
5. La integración de Spring AI como abstracción vendor-neutral para búsqueda semántica, RAG y fraud detection.

Más allá de su valor como entregable académico, este proyecto sirve al autor como blueprint arquitectónico para posiciones Senior/Arquitecto y como caso de estudio replicable en entrevistas técnicas. La mayoría de los patrones son aplicables directamente al trabajo diario en NFD S.A.

Matías Martínez

SRE & Backend Engineer — NFD S.A.

Bootcamp Arquitecturas Backend a Gran Escala con Java — Código Facilito 2026